# 37. gRPC and Protobuf Intro

## 37. gRPC and Protobuf Intro

**Phase:** Phase 6 — Systems and infrastructure
**Estimated time:** 2 days
**Slug:** `37-grpc-protobuf`

### Learning objectives

- Explain protobuf IDL and codegen
- Sketch a gRPC service in Go
- Compare gRPC vs JSON HTTP
- Handle deadlines via context

### Prerequisites

Complete prior numbered lessons. Type every example yourself — reading Go is not writing Go.

### Why this matters

gRPC and protobuf give typed, efficient RPCs across languages. Go is a first-class citizen in that ecosystem.

This lesson is a building block for later modules in the Goodway path. If you only skim, the exercises will feel like busywork; if you type the examples and narrate them aloud, the same patterns will reappear in HTTP handlers, concurrency, and AI service glue.

### Core concepts

- Protobuf defines messages and services in .proto files.
- protoc + plugins generate Go types and stubs.
- gRPC supports unary and streaming RPCs.
- Errors use status codes, not only ad-hoc strings.
- Version your APIs; avoid breaking field numbers.

Hold these in working memory while you code. Each bullet should map to something you can demonstrate with a three-line program or a failing compile error.

### Worked Go examples

Create a scratch module and type this yourself (do not paste blindly):

```
mkdir -p /tmp/gw-grpc && cd /tmp/gw-grpc
go mod init example.com/scratch
```

Then create `main.go`:

```
package main
```

```go
import "fmt"

// Conceptual: generated code looks like this shape.
type HelloRequest struct{ Name string }
type HelloReply struct{ Message string }

func SayHello(req HelloRequest) HelloReply {
    return HelloReply{Message: "hello " + req.Name}
}

func main() { fmt.Println(SayHello(HelloRequest{Name: "Ada"}).Message) }
```

Run it:

```
go run .
```

Expected output:

```
hello Ada
```

If your output differs, diff character-by-character — especially whitespace and types. Then change one line deliberately and predict the new output before running.

### Mini-lab (`go run` steps)

1. Skim a .proto example online and map fields to Go structs.
2. Implement a pure Go stand-in for a unary RPC handler.
3. List three gRPC status codes you would use.

Write down one sentence explaining *why* the output matches the code before you move on. Time-box the lab to 15–25 minutes; struggle is part of encoding the skill.

## Curriculum notes (expanded from the compact draft)

### Why gRPC?

- Strongly typed contracts (`.proto`)
- Efficient binary encoding
- Streaming RPCs
- First-class deadlines/status codes

### Flow

1. Write `service.proto`
2. Generate Go with `protoc` / `buf`
3. Implement server interface
4. Dial with client stubs

### vs REST/JSON

Use gRPC for internal service meshes and streaming; REST/JSON for public browser-facing APIs often still wins.

Contexts carry deadlines across gRPC calls — wire them end-to-end.

## Failure modes to watch while coding

- Exposing gRPC directly to browsers without grpc-web/gateway
- Ignoring status codes (use codes.NotFound etc.)
- Huge protobuf messages without pagination

When a test fails, ask which failure mode you hit before changing code at random.

## Idioms vs Python pitfalls

- grpcio similar concepts; generated stubs differ.
- dict payloads vs typed protobuf messages.
- Field numbers are the compatibility contract — like JSON keys but stricter.

When you catch yourself reaching for a Python habit, pause and rewrite the Go way once on purpose — even if both would "work". The goal is a Go-shaped intuition.

## Going deeper

Start with unary RPCs; add streaming when a real need appears.

## Practice checklist

Before the exercise, confirm you can:

1. Explain "gRPC and Protobuf Intro" aloud in under two minutes without notes.
2. Re-type the worked example from memory and match the expected output.
3. Name the Go standard library packages touched by this lesson.
4. List one production bug this lesson's rules prevent.
5. Contrast the Go approach with the Python instinct listed above.
6. Write a three-line cheat sheet: one idiom, one pitfall, one command to remember.

If any item fails, re-run the mini-lab once more, then attempt the exercise tests.

## Transfer to production

Ask where this shows up outside toys: config loading, API handlers, data pipelines, CI scripts, or on-call debugging. Write one concrete two-sentence scenario tying the worked example to that scenario. Tomorrow, when you rewrite the exercise from memory, check whether the code still fits that framing.

## Common mistakes (especially from Python instincts)

- Exposing gRPC directly to browsers without grpc-web/gateway
- Ignoring status codes (use codes.NotFound etc.)
- Huge protobuf messages without pagination

## Exercises

Work in `exercises/37-grpc-protobuf/`. Read that folder's `README.md` before you code.

**What you will build**

Implement the sketch/stub service method the tests call.

**Study loop**

- **A (Drill):** make unit tests pass with the simplest correct code.
- **B (Apply):** extend with an edge case called out in the exercise README.
- **C (Challenge):** rewrite your solution from memory the next day without looking.

**Do not open `solutions/37-grpc-protobuf/` until you have an honest failing attempt.**

## Self-check

Answer without scrolling up:

1. What does codegen produce?
2. When prefer gRPC?
3. How deadlines propagate?

## Further reading

- https://grpc.io/docs/languages/go/quickstart/